Programación

Recuperación 2ª Evaluación

Problema 1. (4 ptos.) Diseña las siguientes clases:

Clase **Equipo**:

- Atributos: "nombre" de tipo String y "presupuesto" (millones de euros) de tipo double.
- (0,25 ptos.) Constructor que recibe valores para los dos atributos.

Clase **Futbolista** con las siguientes características:

- (0,25 ptos.) Atributos: "nombre" de tipo String, "edad" de tipo int, "posición" de tipo String, dos booleanos "titular" y "reserva", que indican si el futbolista juega de titular o si por el contrario está en la reserva, y "equipo" de tipo Equipo que hace referencia al equipo al que pertenece el futbolista.
- (0,25 ptos.) Constructor para dar valor a los atributos "nombre", "edad" y "posición", y constructor por defecto.
- (0,25 ptos.) Implementa el método *toString()* para que muestre la información de un futbolista con el siguiente formato:

  "El futbolista " + <nombre> + " tiene " + <edad> + " años y juega de " + <posicion> + " en el equipo " + <nombre del equipo>.

  Donde los parámetros entre <> deben tomar los valores apropiados para cada objeto.
- (0,25 ptos.) Método público *void aLaReserva ()* que pasa al futbolista a la reserva.
- (0,25 ptos.) Método público *void aTitular ()* que convierte al futbolista en titular.
- (0,25 ptos.) Método público *boolean equals (Futbolista f)* que determine si dos objetos Futbolista representan al mismo jugador (son iguales).

  NOTA: Dos futbolistas son iguales si tienen el mismo nombre, edad y juegan en la misma posición.
- (0,25 ptos.) Método público *int compareTo (Futbolista f)* que compara dos objetos FUTBOLISTA teniendo en cuenta su edad.
- (0,25 ptos.) Método público *boolean jugarConLasManos* que devolverá verdadero si el FUTBOLISTA puede jugar con las manos y falso en caso contrario

  NOTA: Un futbolista sólo puede jugar con las manos si es Portero.

Clase **Jugador** que hereda de Futbolista:

- Atributos: "golesMarcados" y "dorsal", ambos de tipo int.
- (0,25 ptos.) Constructor que de valor a los atributos y constructor por defecto.
- (0,25 ptos.) Implementa el método *toString()* para que muestre la información de un jugador con el siguiente formato:

  "El futbolista " + <nombre> + " tiene " + <edad> + " años y juega de " + <posición> + " en el equipo " + <nombre del equipo> + " con el dorsal " + <dorsal> + " . Ha marcado " + <goles> + " goles".

Donde los parámetros entre <> deben tomar los valores apropiados para cada objeto.

Clase **Portero** que hereda de Futbolista:

- Es similar a la clase Jugador, con los mismos atributos, salvo que ahora el atributo "golesMarcados" se llama "golesRecibidos", la "posición" deberá ser siempre portero y como es lógico puede jugar con las manos.
- (0,25 ptos.) Ahora el método *toString()* mostrará la información de un portero con el siguiente formato:

  "El futbolista " + <nombre> + " tiene " + <edad> + " años y juega de " + <posición> + " en el equipo " + <nombre del equipo> + " con el dorsal " + <dorsal> + " . Ha recibido " + <goles> + " goles".

Clase **Liga**, que será la raíz de la aplicación. Crea en ella los siguientes métodos:

- (0,25 ptos.) *boolean esPortero (Futbolista f)* que determina si un Futbolista es Portero o no.
- (0,25 ptos.) *void mostrarFicha (Futbolista f)* que muestra los datos del futbolista.
- (0,5 ptos.) *void vender (Futbolista f, double precio)*, que transfiere al Futbolista únicamente si está en la reserva, en cuyo caso deja de estar ligado a su Equipo y el presupuesto del Equipo se aumentará en el precio indicado.

Problema 2. (4 ptos.)

Implementa un diccionario de palabras utilizando un **mapa**. Cada palabra es única y puede tener uno o varios significados, por lo que se utilizará un **ArrayList** para almacenarlos.

Realiza los siguientes métodos:

- (1 pto.) *addPalabra(String palabra, String significado)*: método que almacena la palabra con el significado recibidos como parámetros, si la palabra no existe en el diccionario. En el caso de que la palabra ya esté almacenada en el diccionario entonces añade el nuevo significado a los ya existentes.
- (1 pto.) *getSignificado(String palabra)*: método que devuelve el PRIMER significado de la palabra recibida como parámetro. En el caso de que la palabra no exista en el diccionario, devolverá -1.
- (1 pto.) *getSignificados(String palabra)*: método que devuelve TODOS los significados de la palabra recibida como parámetro. En el caso de que la palabra no exista en el diccionario, devolverá null.
- (1 pto.) indice(): método que devuelve un listado de todas las palabras del diccionario ordenadas alfabéticamente, mostrando para cada palabra el nº de significados de que dispone entre paréntesis.

Incorpora al programa alguna prueba de tu código. Por ejemplo:

```
N° de palabras a introducir en el diccionario: 2
--------------------------------------------------------------------------
Palabra-1: examen
Significado [ENTER para terminar]: Observación atenta y cuidadosa de
alguien o algo.
Significado [ENTER para terminar]:  Prueba escrita u oral.
Significado [ENTER para terminar]:
Palabra-2: pc
Significado [ENTER para terminar]: Sigla de personal computer.
Significado [ENTER para terminar]:
--------------------------------------------------------------------------
Indice de palabras:
        examen      (2)
        pc          (1)
Significado principal de: examen
        Observación atenta y cuidadosa de alguien o algo.
Significados de: examen
        Observación atenta y cuidadosa de alguien o algo.
        Prueba escrita u oral.
```